

Y-DUDE

TECHNISCHES NOTFALLHANDBUCH

Production

Version 1.0

Erstellt: 14.09.2026

Grundlage: aktueller Production-Codestand, Lovable-Cloud-Instanz, Betriebsdokumentation

Legende der Statusbewertung

- sicher / stabil
- nur nach Prüfung
- nicht durchführen
- nicht getestet / nicht bewertet

Farbige Punkte ersetzen die Symbole des Quelldokuments – PDF-Schriften stellen Emoji nicht dar.

Dieses Dokument enthält keine Secrets, Schlüssel oder Passwörter, nur Variablennamen.

Für die Erstellung wurden keine Änderungen an Production vorgenommen.

Vertraulich – interner Betriebsgebrauch

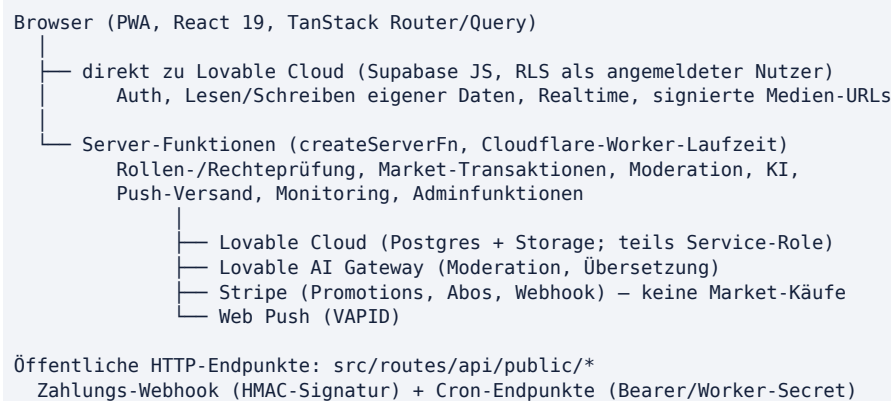
Version: 1.0 **Erstellt:** 14.09.2026 **Grundlage:** aktueller Production-Codestand dieses Repositories, die aktive Lovable-Cloud-Instanz (Schema, Rechte, Zeitpläne, Speicher) und die vorhandene Betriebsdokumentation unter docs/.

Geltungsregeln für dieses Dokument

- Es beschreibt ausschließlich Bestandteile, die im aktuellen Projekt nachweisbar sind. Nicht belegbare Punkte stehen in Abschnitt 19 („Offene Dokumentationslücken“) oder sind mit **NICHT DOKUMENTIERT / EXTERN ZU PRÜFEN** gekennzeichnet.
- Es enthält keine Secrets, Schlüssel oder Passwörter – nur Variablennamen.
- Es wurden für dieses Dokument keine Änderungen an Production, Datenbank, Rechten, Policies oder Code vorgenommen (rein lesende Analyse).
- Ergänzende Dokumente: docs/ARCHITEKTUR.md, docs/RUNBOOK_CRITICAL_OPS.md, docs/RUNBOOK_INCIDENT.md, docs/BETRIEB_LOGS_BACKUPS_VORFALL.md.

1. SYSTEMÜBERSICHT

1.1 Gesamtbild



1.2 Bestandteile im Einzelnen

Bestandteil	Ort / Technik	Zweck	Abhängigkeiten	Kritische Funktion	Fehlerauswirkung
Frontend	src/routes/**, src/components/**, React 19, Tailwind 4	Darstellung, Navigation, PWA	Browser, Bundle-Auslieferung	Feed, Messenger, Market-UI	Weißer Seite, Teilbereiche unbenutzbar
SSR / Worker-Entry	src/server.ts, src/start.ts, Vite 8 + Nitro, Cloudflare-Worker	Serverseitiges Rendern, Fehler-Middleware, CSRF, Metriken, Public-Cache	Worker-Laufzeit, Supabase	Erstauslieferung jeder Seite	5xx auf allen Routen (P0)
Server-Funktionen	src/lib/*.functions.ts → src/lib/*.server.ts	Typisierte RPC vom Client zum Server	attachSupabaseAuth in src/start.ts	Market-Transaktionen, Admin, Moderation, Push	Aktionen schlagen fehl, oft 401
Öffentliche Endpunkte	src/routes/api/public/**	Cron-Läufe + Stripe-Webhook	Worker-Tokens, Stripe-Signatur	Moderation, Push, Zähler, Retention, Health	Hintergrundläufe stehen, Zahlungsstatus veraltet
Auth	Lovable Cloud Auth; src/lib/auth.function.s.ts, src/integrations/supabase/*	Registrierung, Anmeldung, Session, Reset	Supabase Auth, E-Mail-Versand	Zugang zur App	Niemand kann sich anmelden (P1)
Datenbank	Postgres (Lovable Cloud), ~128 Tabellen im Schema public	Alle Nutz- und Betriebsdaten	RLS + GRANTS, SECURITY-DEFINER-Helfer	Feed, Messenger, Market, Rollen	„permission denied“, leere Listen, App teilweise tot

Bestandteil	Ort / Technik	Zweck	Abhängigkeiten	Kritische Funktion	Fehlerauswirkung
RPCs	z. B. mark_conversation_read, market_start_transaction, market_complete_transaction, market_accept_offer, flush_counter_events, has_role	Atomare, rechtgeprüfte Operationen	SECURITY DEFINER + gezielte GRANTS	Lesestand, Kaufabschluss, Rollenprüfung	Statusinkonsistenzen, Rechtefehler
Storage	Supabase Storage, ein privater Bucket media	Bilder, Audio, Video, Avatare, Cover, Originale, Varianten	signierte URLs, can_read_media()	Medien in Feed/Profil/Chat/Market	Bilder/Audio fehlen, Uploads scheitern
Realtime	Supabase Realtime, Channels in src/lib/social.tsx	Live-Nachrichten, Presence, Tippen, Benachrichtigungen	Websocket, RLS	Messenger-Live-Verhalten	Nachrichten erst nach Neuladen (P2)
AI Gateway	LOVABLE_API_KEY, Moderation/Übersetzung	Inhaltsprüfung, Übersetzungen	externer Dienst	Moderationslauf	Beiträge bleiben pending, keine Übersetzung
Zahlungen	Stripe (src/lib/stripe.server.ts, src/routes/api/public/payments/webhook.ts)	Promotions, Business-/Creator-Abos	Stripe, Webhook-Signatur	Abo-/Promotion-Status	Abos/Promotions nicht aktiviert (P2)
Web Push	src/lib/push.server.ts, public/push-sw.js, VAPID	Benachrichtigungen	VAPID-Schlüssel, Cron push-run	Zustellung	Keine Push-Nachrichten (P2/P3)
Deployment	Lovable Publish; Build vite build; Gatebun run verify	Veröffentlichung	Build-Erfolg, Migrationsstand	Auslieferung des Stands	Fehlerhafter Stand live (P0/P1)
Migrationen	drizzle/migrations/0000...0038*.sql (autoritativ) + historische supabase/migrations/**	Schema, Rechte, Funktionen, Trigger	Drizzle-Kit, LOVABLE_DB_MIGRATION_URL	Schemastand	Fehlende GRANTS/Policies → Rechtefehler
Service-Role-Zugriff	src/integrations/supabase/client.server.ts (supabaseAdmin)	Privilegierte Serveroperationen	Server-Schlüssel in der Plattform-Secret-Verwaltung	Market-Transaktionen, Webhook, Ops-Events	Serverpfade laufen als anon → „permission denied“
Observability	src/lib/ops-monitor.server.ts, error-capture.ts, runtime-metrics.server.ts, /admin/health	Ereignisse, Vorfälle, Alarmer, Kennzahlen	ops_events, ops_incidents	Früherkennung	Störungen bleiben unentdeckt

1.3 Umgebungen

Einzige Quelle der Wahrheit: src/lib/environment.shared.ts (Server: src/lib/environment.server.ts, APP_ENV hat Vorrang).

Umgebung	Hostnamen	Zahlungsmodus
development	localhost, 127.0.0.1, *.local	Sandbox
staging	Vorschauadressen (id-preview-*, *-dev.lovable.app)	Sandbox
production	y-dude.com, www.y-dude.com, y-dude.lovable.app	Live

Wichtig: Datenbank und Storage sind zwischen Vorschau und veröffentlichter Website **geteilt** (eine Cloud-Instanz). Jeder Schreibvorgang aus der Vorschau wirkt auf Produktionsdaten. Unbekannte Hosts gelten immer als Staging.

1.4 Environment-Variablen (nur Namen)

Client (Build-Zeit, öffentlich): VITE_SUPABASE_URL, VITE_SUPABASE_PUBLISHABLE_KEY, VITE_SUPABASE_PROJECT_ID, VITE_PAYMENTS_CLIENT_TOKEN, VITE_ADSENSE_*.

Server (nur in Handlern lesen):

Name	Zweck	Fehlt er, dann ...
SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY	serverseitige Datenzugriffe	Server-Funktionen scheitern
Server-Schlüssel für supabaseAdmin (Plattform-gebunden)	privilegierte Operationen	Market/Webhook/Ops: „permission denied“
APP_ENV	Umgebung erzwingen	Erkennung nur über Hostnamen
LOVABLE_API_KEY	Moderation, Übersetzung	Moderation/Übersetzung fällt zurück
STRIPE_LIVE_API_KEY	Live-Zahlungen	kein Live-Checkout
MODERATION_CRON_TOKEN	Sammel-Token aller Cron-Endpunkte	alle Cron-Läufe → 401
PUSH_CRON_TOKEN, COUNTERS_CRON_TOKEN, METRICS_CRON_TOKEN, RETENTION_CRON_TOKEN, OPS_HEALTH_CRON_TOKEN, BETA_LAUNCH_CRON_TOKEN	Einzel-Token je Endpunkt	Ausweichpfad MODERATION_CRON_TOKEN
VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY, VAPID_SUBJECT	Web Push	keine Push-Nachrichten
CLOUDFLARE_TURNSTILE_SITE_KEY / _SECRET_KEY	Bot-Schutz	Turnstile-Prüfung inaktiv
OPS_ALERT_WEBHOOK_URL, OPS_ALERT_WEBHOOK_URL_2	Alarmzustellung	Alarmer nur in Protokoll//admin/health
OPS_HEARTBEAT_URL	Totmannschalter	kein Alarm bei Totalausfall
MASTER_ADMIN_PASSWORD	Notzugang Adminbereich	nur regulärer Rollenweg
ALLOW_TEST_FEATURES_IN_PRODUCTION	Testfunktionen (Standard: aus)	-
LOVABLE_DB_MIGRATION_URL	Drizzle-Migrationen	Migrationen nicht ausführbar

Secrets werden ausschließlich über die Lovable-Secret-Verwaltung gesetzt, niemals im Repository. Der Service-Role-Schlüssel und das Datenbankpasswort sind auf Lovable Cloud **nicht auslesbar**.

1.5 Aktive Zeitpläne (Stand: Live-Datenbank, 14.09.2026)

Job	Takt	Wirkung
post-moderation-worker	jede Minute	/api/public/moderation-run
y-dude-counter-flush	jede Minute	flush_counter_events
y-dude-push-run	jede Minute	/api/public/push-run
y-dude-ops-health	5 Minuten	/api/public/ops-health-run + Heartbeat
refresh-connection-suggestions	10 Minuten	Connection-Vorschläge
y-dude-exclusive-drop-maturation	stündlich (:07)	Reifung exklusiver Drops
y-dude-retention-run	täglich 03:17 UTC	/api/public/retention-run

2. NOTFALLKLASSEN

Klasse	Lage	Erkennungsmerkmal	Erste Reaktion
P0	Y-Dude vollständig nicht erreichbar	/ liefert 5xx oder nichts; alle Routen betroffen	sofort, alles zurückstellen
P1-A	Anmeldung/Auth ausgefallen	/auth ohne Session, Anmeldung schlägt für alle fehl	sofort
P1-B	Datenbank / Data API ausgefallen	überall leere Daten oder „permission denied“/Timeouts	sofort
P1-C	Messenger ausgefallen	Nachrichten nicht ladbar/sendbar	sofort
P1-D	Market ausgefallen	Artikel/Transaktionen fehlerhaft, Statusinkonsistenz	sofort
P1-E	Storage/Uploads ausgefallen	Bilder/Audio fehlen, Uploads scheitern	sofort
P1-S	Sicherheitsvorfall (bestätigt)	Secret-Leak, Fremdzugriff, RLS-Lücke	sofort, Spuren erhalten
P2-A	einzelne Funktion fehlerhaft	begrenzter Bereich (z. B. Arena, Werbung)	am gleichen Arbeitstag
P2-B	Performance massiv reduziert	p95 dauerhaft weit über Normalwerten, Timeouts	am gleichen Arbeitstag
P2-C	Realtime-Probleme	Nachrichten erst nach Neuladen, Presence tot	am gleichen Arbeitstag
P3	nichtkritische UI/UX-Fehler	Darstellung, Text, Einzelfall	normaler Arbeitsablauf

Technische Zuordnung im Code: ops_events.severity (info/warning/critical) und ops_events.area (api, database, rpc, auth, payments, webhook, push, performance, security) aus src/lib/ops-monitor.shared.ts. Die Betriebsklasse

oben ist die menschliche Bewertung und kann höher liegen.

3. SOFORTMASSNAHMEN (Standardablauf für jeden Notfall)

1. **Problem bestätigen** – eigene Reproduktion, URL, Zeitpunkt, Umgebung (Vorschau oder Production) festhalten. Nicht auf eine Einzelmeldung hin handeln.
2. **Umfang feststellen** – / (SSR), /auth (Anmeldung), Feed, Messenger, Market einzeln prüfen. Alle Nutzer oder nur einzelne?
3. **Production-Daten schützen** – keine Löschläufe, keine DELETE/UPDATE „zum Aufräumen“, retention-run nicht anstoßen. Vorschau schreibt in dieselbe Datenbank: auch dort keine Experimente mit Schreibzugriff.
4. **Logs prüfen** – /admin/health (Ereignisse 24 h, Vorfälle), Serverlogs des betroffenen Deployments (Preview und Published getrennt), Backend-Logs der Cloud, bei Zahlungen zusätzlich das Stripe-Dashboard.
5. **Betroffene Komponente identifizieren** – Entscheidungsbaum in Abschnitt 9.
6. **Keine unkontrollierten Änderungen** – kein Publish „auf Verdacht“, keine Policy-/Grant-Änderung ohne Diagnose, keine Secret-Rotation ohne Anlass.
7. **Ursache eingrenzen** – letzte Veröffentlichung, letzte Migration, externer Dienst? Reproduktion möglichst in der Vorschau, rein lesend.
8. **Recovery durchführen** – kleinste wirksame Maßnahme: Funktion abschalten statt App abschalten (/admin/ads, /admin/livetest), Fix + Publish, Rückkehr zu einem geprüften Codestand (Abschnitt 12).
9. **Funktion verifizieren** – Rauchtest: / lädt → Anmeldung → Feed inkl. Medien → Profil/Connections → Nachricht senden + Lesestand → Market-Liste → /admin/health ohne neue kritische Ereignisse.
10. **Vorfall dokumentieren** – ops_incidents.note, bei P0/P1 zusätzlich docs/incidents/JJJJ-MM-TT-kurzname.md nach der Vorlage in docs/RUNBOOK_INCIDENT.md §4. Keine personenbezogenen Daten.

Nie als Standardmaßnahme: DROP, TRUNCATE, DELETE ohne WHERE, Massen-UPDATE auf Statusspalten, Deaktivieren von RLS, Löschen von Migrationen, Löschen von Storage-Objekten.

4. AUTHENTICATION

4.1 Realer Ablauf im Code

Vorgang	Datei / Funktion	Besonderheit
Anmeldung	src/lib/auth.functions.ts → signInWithCaptcha	serverseitig; Turnstile-Gate vorgeschaltet, derzeit zentral abgeschaltet über src/lib/turnstile-flag.ts; nutzt createPublicServerClient() aus src/lib/auth-public.server.ts; Ergebnisse ok / unconfirmed / invalid / captcha
Registrierung	src/lib/auth.functions.ts → signUpWithCaptcha	Zod-Validierung (E-Mail, Passwort ≥ 8, Username <code>^[a-zA-Z0-9_]{3,24}\$</code> , Geburtsdatum Pflicht/Jugendschutz); Ergebnisse u. a. confirm, underage, username_taken, username_blocked, email_taken, rate_limited
Bestätigungsmail erneut	resendConfirmationEmail (gleiche Datei)	neutrale Antwort, cooldown bei 429
Passwort-Reset	resetPasswordForEmail in src/lib/auth.functions.ts; Einlösen in src/routes/reset-password.tsx (onAuthStateChanged, danach signOut)	Reset-Link führt auf eine öffentliche Route
Session (Client)	src/integrations/supabase/client.ts (generiert), src/lib/use-session.ts, src/lib/session-bootstrap.ts	Session im Browser-Storage; onAuthStateChanged in src/routes/__root.tsx und src/lib/data.tsx
Routenschutz	src/routes/_authenticated/route.tsx	ssr: false, beforeLoad prüft Nutzer, sonst Weiterleitung nach /auth
Token an Server-Funktionen	attachSupabaseAuth in src/start.ts (functionMiddleware)	fehlt die Registrierung, antworten alle geschützten Server-Funktionen 401
Serverseitige Prüfung	requireSupabaseAuth (src/integrations/supabase/auth-middleware.ts)	liefert context.supabase, userId, claims

Vorgang	Datei / Funktion	Besonderheit
Rollen / Admin	user_roles (Enum app_role) + admin_owners; Prüfung über has_role(auth.uid(), 'admin'), is_admin_owner(); Notzugang MASTER_ADMIN_PASSWORD; Protokoll admin_audit_log	Rollen liegen nie am Profil
Registrierungs-Gesundheit	src/lib/registration-health.server.ts, /admin/registration-check, Tabellen registration_events, registration_health_checks	datenschutzarme Ereignisse (keine E-Mail, keine User-ID, keine IP)

4.2 Typische Fehlerbilder

Was sehe ich?	Wo prüfe ich?	Was darf ich ändern?	Wie verifiziere ich?
Alle Anmeldungen scheitern, UI meldet „ungültig“	Serverlogs ([auth] ...), Auth-Logs im Backend, /admin/registration-check	● nichts blind – erst prüfen, ob Auth-Dienst selbst gestört ist	Testanmeldung mit einem bekannten Konto; Session wird gesetzt
Jede geschützte Aktion antwortet 401, Anmeldung selbst geht	src/start.ts (functionMiddleware enthält attachSupabaseAuth?), Serverlogs	● Registrierung der Middleware wiederherstellen, dann veröffentlichen	Beliebige geschützte Server-Funktion (z. B. Nachricht senden) funktioniert
„email_not_confirmed“ / Nutzer kommt nicht rein	Auth-Einstellungen (E-Mail-Bestätigung), E-Mail-Versand/Absenderdomain	● erneutes Senden der Bestätigung anstoßen	Nutzer meldet sich nach Bestätigung an
Registrierung bricht ohne Meldung ab	registration_events (event, cause), Serverlogs, Turnstile-Schalter	● Turnstile-Flag/Keys prüfen	Neue Testregistrierung durchläuft; Ereignis mit event=success
Reset-Link führt zu leerer Seite	src/routes/reset-password.tsx, Redirect-URL-Konfiguration	● Redirect-Ziel muss öffentliche Route sein	Reset komplett durchlaufen, danach Anmeldung
Adminbereich verweigert Zugang	user_roles (Rolle vorhanden?), admin_owners, admin_audit_log	● keine Rollen „schnell“ per SQL setzen – über den vorgesehenen Weg	/admin/health lädt für die berechnete Person
Endlose Weiterleitung auf /auth	src/routes/_authenticated/route.tsx (muss ssr: false bleiben)	● kein zusätzlicher Auth-Gate auf SSR-Routen	Harter Reload einer geschützten Route bleibt angemeldet

5. DATENBANK / SUPABASE

5.1 Tabellenbereiche (Auszug, real vorhanden)

Bereich	Tabellen
Profile/Identität	profiles, user_roles, admin_owners, identity_policy, reserved_usernames, user_bans, user_warnings
Inhalte	posts, comments, post_likes/saves/shares/views, post_originals, hashtags, post_hashtags, post_translations
Messenger	conversations, conversation_members, messages, message_translations, chat_slang_tags
Market	market_items, market_images, market_offers, market_transactions, market_transaction_events, market_transaction_secrets, market_shipping, market_payment_records, market_payment_webhook_events, market_refunds, market_disputes, market_promotions, market_seller_profiles, market_searches, market_favorites
SlangTags/Arena	slang_tags, slang_definitions, slang_tag_grants, slang_tag_*, arena_*
Sozial	follows, connections, connection_suggestions, notifications, push_subscriptions, notification_jobs
Channels	channels, channel_members, channel_follows, channel_bans, channel_categories
Feed/Interessen	feed_signals, feed_score_cache, feed_learned_weights, interest_*, user_interest_scores, counter_events
Medien	media_video_assets, media_variant_jobs
Moderation	post_moderation_jobs, moderation_actions, moderation_appeals, content_moderation_log, reports
Betrieb	ops_events, ops_incidents, admin_audit_log, account_security_events, registration_events, registration_health_checks, feedback

5.2 RLS-Konzept

- Jede Tabelle im Schema public hat RLS aktiv **und** explizite GRANTS. Fehlt der GRANT, meldet die App „permission denied“ – trotz korrekter Policy.
- Reihenfolge bei neuen Tabellen: CREATE TABLE → GRANT → ENABLE ROW LEVEL SECURITY → CREATE POLICY.
- Rollen ausschließlich in user_roles; Prüfung über has_role().
- Sichtbarkeit über SECURITY-DEFINER-Helfer statt verschachtelter Policies: can_view_post, can_view_profile, can_see_profile_field, can_read_media, is_conversation_member, is_channel_moderator, are_connected, can_see_arena_submission u. a.
- **Sensibelste Stelle des Projekts:** eine Änderung an diesen Helfern wirkt auf viele Policies gleichzeitig. Nach jeder Änderung bunx vitest run tests/rls-policy-contract.test.ts und Sicherheits-Scan.

5.3 Wichtige RPCs

RPC	Zweck	Rechte
mark_conversation_read(_conversation_id)	setzt messages.read_at (nur fremde) + conversation_members.last_read_at; prüft Mitgliedschaft	SECURITY DEFINER; authenticated, service_role; entzogen für PUBLIC/anon (Migration 0035)
market_start_transaction(...)	sperrt Artikel (FOR UPDATE), prüft active/Kaufbarkeit, Preis-Snapshot, Gebühr, erzeugt Transaktion + market_transaction_secrets, setzt Artikel auf reserved	SECURITY DEFINER; kein anon/authenticated-EXECUTE
market_complete_transaction(_tx_id, _seller_id)	sperrt Transaktion und Artikel, setzt atomar completed + sold; Fehler transaction_not_found, cancelled, already_sold	SECURITY DEFINER; EXECUTE nur service_role (Migrationen 0037, 0038)
market_accept_offer(_offer_id)	Angebot annehmen	SECURITY DEFINER, service_role
flush_counter_events(_max)	Zähler-Ereignisse verdichten	Cron
has_role(_user_id, _role), is_admin_owner(_user_id)	Rollenprüfung	Basis aller Adminpolicies
bootstrap_user_state()	Startzustand nach Anmeldung	-
can_read_media(_object_name)	Storage-Leserecht	von Storage-Policies genutzt

5.4 Trigger (Auszug)

guard_transaction_events (Transaktionsprotokoll strikt append-only), guard_profile_identity / guard_profile_internal_fields, guard_reserved_username, enforce_write_rate_limit, enforce_report_rate_limit, enqueue_notification_push, bump_conversation_activity, channel_owner_membership, enforce_campaign_media_owner, guard_admin_role_changes, lapse_pending_drops_on_subscription_change.

5.5 Migrationen

- **Autoritativ:** drizzle/migrations/0000...0038*.sql (Drizzle-Kit, Journal in drizzle/migrations/meta), DB-Verbindung über LOVABLE_DB_MIGRATION_URL.
- Historisch zusätzlich supabase/migrations/** (chronologisch, gewachsen).
- Bestehende Migrationen **niemals** nachträglich ändern – immer eine neue.
- Migrationen additiv denken: neue Spalten nullable oder mit Default, keine Blind-Drops; Vorschau und Production teilen dasselbe Schema.

5.6 Kritische Fremdschlüssel und Transaktionslogik

- market_transactions → market_items, Käufer/Verkäufer (Profile); market_transaction_secrets → market_transactions.
- conversation_members → conversations; messages → conversations (+ optional market_item_id, market_offer_id).
- Partieller UNIQUE-Index uniq_market_tx_open_item auf market_transactions(item_id) für offene Status – verhindert zwei parallele offene Vorgänge zum selben Artikel.
- market_payment_webhook_events mit UNIQUE(provider, event_id) – Idempotenz des Zahlungs-Webhooks.

5.7 Service-Role-Zugriffe

supabaseAdmin aus src/integrations/supabase/client.server.ts, geladen innerhalb der Handler. Verwendet in src/lib/market-tx.server.ts (alle Statusoperationen), src/lib/market-chat.server.ts, src/routes/api/public/payments/webhook.ts (Event-Claim), src/lib/ops-monitor.server.ts (Ereignisse/Vorfälle),

Push- und Moderationspfade. Regel: nie für gewöhnliche Lesezugriffe, nie zur Entscheidung über Adminrechte.

Keine Security-Policies im Notfall verändern. Rechtefehler werden zuerst als fehlender GRANT untersucht, nicht durch Aufweichen von Policies „gelöst“.

6. MESSENGER-NOTFALL

6.1 Realer Ablauf

Chat öffnen
 → conversations (Metadaten) + conversation_members (Mitgliedschaft, last_read_at)
 → messages (Seitenweise, 30 pro Block)
 → Lesestand: RPC mark_conversation_read (entprellt, 2 s)
 → Realtime: postgres_changes + Presence + Broadcast

6.2 Beteiligte Dateien

Datei	Rolle
src/lib/social.tsx	Kern: Laden von Konversationen/Nachrichten, Senden, Lesestand, Realtime-Abos, Presence
src/lib/social-context.ts	Context-Anbindung (useSocial())
src/components/Messenger.tsx	Oberfläche, Chatliste, Eingabe, Angebote
src/lib/session-bootstrap.ts	gebündeltes Erstladen
src/lib/messenger-view.ts	reine Ansichtslogik (Kategorien, Reset)
src/lib/market-chat.server.ts, src/lib/market.functions.ts	Market-Kontext und Angebote im Chat
src/lib/push-active-chat.ts	meldet offenen Chat an den Service Worker (keine Push bei offenem Chat)
drizzle/migrations/0035_hardening_m4_mark_conversation_read.sql	RPC-Definition Lesestand

6.3 Betroffene Tabellen und RPCs

Tabellen: conversations, conversation_members, messages, chat_slang_tags, message_translations, zusätzlich market_offers / market_items für Market-Chats. RPCs: mark_conversation_read, market_accept_offer.

6.4 Realtime-Kanäle

- ydude-social-<userId> – postgres_changes auf connections, conversation_members, messages (INSERT/UPDATE), notifications
- chat-<conversationId> – Broadcast „tippt...“, höchstens 60 Kanäle
- presence-u-<userId> – Presence, Peers begrenzt auf 80

6.5 Logs

Serverlogs und Browserkonsole nach [social] ... filtern: sendMessage, markConversationRead, connections_fetch_error, openDirectChat. Zusätzlich ops_events mit area=rpc (RPC-Fehler) bzw. area=database.

6.6 DB- vs. API- vs. Realtime-Problem unterscheiden

Beobachtung	Wahrscheinlicher Bereich	Nächster Schritt
Nachrichtenliste bleibt leer, Fehler „permission denied“	Datenbank (GRANT/Policy)	Rechte auf messages, conversation_members prüfen
Senden schlägt fehl, Konsole [social] sendMessage	Data API / Policy	Insert-Policy auf messages, Rate-Limit-Trigger prüfen
Nachrichten erscheinen erst nach Neuladen	Realtime	Websocket-Verbindung, Kanalgrenzen (60/80), Realtime-Status im Backend
Lesestand bleibt stehen	RPC mark_conversation_read	Fehlerlog, EXECUTE-Rechte (authenticated)
Nur Market-Chats betroffen	Server-Funktionen market.functions.ts	Serverlogs, requireSupabaseAuth, Service-Role-Zugriff
Alles langsam, aber vollständig	Performance	Abschnitt 9

Keine manuellen Änderungen an `messages` oder `conversation_members` als erste Maßnahme – verlorene oder doppelte Nachrichten zuerst über Zählungen belegen.

7. MARKET-NOTFALL

7.1 Realer Ablauf (Stand 09/2026: vereinfachter Flow, Abholung, keine In-App-Zahlung für Artikelkäufe)

Artikel anlegen	→ <code>market_items</code> (status = active) + <code>market_images</code>
Angebot	→ <code>market_offers</code> (open) + Chat-Nachricht (kind = market_offer)
Annahme	→ RPC <code>market_accept_offer</code>
Kauf starten	→ RPC <code>market_start_transaction</code> Artikel FOR UPDATE, Prüfung active/buy_now, Preis-Snapshot, Gebühr, <code>market_transactions</code> + <code>market_transaction_secrets</code> , <code>market_items.status</code> = reserved danach serverseitig: ready_for_pickup / not_required
Abschluss	→ RPC <code>market_complete_transaction</code> Transaktion + Artikel FOR UPDATE <code>market_transactions.status</code> = completed <code>market_items.status</code> = sold (atomar, eine Transaktion)

Statuswerte: `market_item_status` = active|reserved|sold|disabled|deleted; `market_transaction_status` = pending, payment_pending, paid, processing, ready_for_pickup, shipped, completed, cancelled, refunded, disputed.

7.2 Wie eine Transaktion korrekt abgeschlossen wird

Es gibt genau **einen** autoritativen Weg – die Datenbankfunktion `market_complete_transaction(_tx_id, _seller_id)`. Sie wird erreicht über:

- `markSold()` in `src/lib/market-tx.server.ts` (Verkäufer bestätigt Übergabe, Ereignis via: "seller_confirmation"), und
- `completeOpenTransactionsForItem()` in derselben Datei, ausgelöst von `setMarketItemStatus("sold")` in `src/lib/market.functions.ts` (Ereignis via: "listing_marked_sold").

Die Artikel-ID stammt immer aus der Transaktion, nie vom Client. Wiederholte Aufrufe sind idempotent (already_sold). Ein Abholcode wird im aktuellen Flow nicht mehr an den Client ausgegeben.

7.3 Konsistenz zwischen `completed` und `sold` prüfen

Rein lesende Prüfungen (keine Änderung):

```
-- A) abgeschlossene Vorgänge, deren Artikel nicht 'sold' ist
select t.id, t.status, i.id as item_id, i.status as item_status
from market_transactions t
join market_items i on i.id = t.item_id
where t.status = 'completed' and i.status <> 'sold';

-- B) verkaufte Artikel ohne abgeschlossenen Vorgang
select i.id, i.status
from market_items i
where i.status = 'sold'
and not exists (select 1 from market_transactions t
               where t.item_id = i.id and t.status = 'completed');

-- C) mehrere offene Vorgänge je Artikel (dürfen nicht existieren)
select item_id, count(*) from market_transactions
where status in ('pending', 'payment_pending', 'paid', 'processing',
               'ready_for_pickup', 'shipped', 'disputed')
group by item_id having count(*) > 1;
```

Befund A: erneuter Aufruf von `market_complete_transaction` über den regulären Serverpfad (`markSold`) ist der richtige Weg – **kein** manuelles UPDATE auf `market_items.status`. Befund B ist zulässig, wenn der Verkäufer den Artikel manuell auf „verkauft“ gesetzt hat, ohne dass ein Vorgang existierte. Befund C wäre ein echter Fehler (der partielle UNIQUE-Index soll das verhindern).

7.4 Duplicate Detection / Idempotenz (real vorhanden)

1. Partieller UNIQUE-Index `uniq_market_tx_open_item` – nur ein offener Vorgang je Artikel.
2. `already_sold`-Prüfung in `market_complete_transaction` – kein doppelter Abschluss.
3. `UNIQUE(provider, event_id)` auf `market_payment_webhook_events` + `claimEvent` im Webhook – kein doppelt verarbeitetes Zahlungseignis.

4. Trigger guard_transaction_events – Ereignisprotokoll append-only.
5. Angebote: keine DB-Unique-Regel; Dedupe in createOffer (vorheriges offenes Angebot desselben Käufers wird auf withdrawn gesetzt).

7.5 Zahlungen im Market

src/routes/api/public/payments/webhook.ts prüft die Stripe-Signatur und ein Umgebungs-Gate (env-Parameter gegen paymentsModeAllowed) und bedient **nur** Promotions sowie Business-/Creator-Abos. Für Artikelkäufe gibt es bewusst keinen Zahlungspfad. Bleibt eine Promotion oder ein Abo inaktiv, ist das ein Webhook-/Signatur-/Umgebungsproblem, kein Transaktionsproblem.

7.6 Diagnose

Beobachtung	Prüfen	Maßnahme
„permission denied for function market_complete_transaction“	ob der Serverpfad tatsächlich mit Service-Role läuft (Serverlogs), EXECUTE-Rechte	Server-Schlüssel/Bindung prüfen; keine Rechte an anon/authenticated vergeben
Artikel bleibt reserved	offener Vorgang vorhanden? Fehler im Abschlusslauf?	Abschluss über den regulären Serverpfad wiederholen
Käufer sieht Artikel weiter als verfügbar	Cache/Client vs. DB-Status	DB-Status per Abfrage feststellen, dann Client neu laden
Doppelter Vorgang	Abfrage C in 7.3	Ursache im Index/Serverpfad suchen, nicht Datensätze löschen
Abo/Promotion nicht aktiv	Stripe-Ereignisse, Webhook-Zustellversuche, market_payment_webhook_events	Webhook-Konfiguration und Umgebungs-Parameter prüfen

8. STORAGE / UPLOADS

8.1 Bucket und Pfade

- **Ein einziger, privater Bucket:** media (bestätigt in der Live-Instanz).
- Pfadschema <userId>/<ordner>/<uuid>.<ext> mit den Ordnern images, audio, avatars, covers, originals, videos, variants (src/lib/media.ts).
- Cache: unveränderliche Auslieferung für alle Ordner außer originals (dort no-store).

8.2 Upload-Flows

Typ	Weg	Prüfungen
Bild (Beitrag, Avatar, Cover, Market)	Client lädt in media, Varianten __t.webp/ __m.webp werden clientseitig erzeugt	src/lib/image-limits.ts, image-crop.ts, cover-crop.ts
Variantenausfall	Backstop ensureVariantsForPath in src/lib/media-variants.server.ts	signierte URL mit Transformation, Quelle max. 25 MB, upsert: false (idempotent)
Video	src/lib/video/video-upload-client.ts lädt hoch, dann Server-Funktion registerVideoUpload (src/lib/video/video-upload.functions.ts)	requireSupabaseAuth, Pfadbesitz-Prüfung, Größe/MIME aus storage.list (nicht vom Client), Containerprüfung per HTTP-Range (src/lib/video/video-file.ts: max. 60 s, 50 MB, MP4/MOV/M4V); bei Fehlschlag wird das Original gelöscht; Ergebnis in media_video_assets
Audio	Upload in audio, Formatprüfung src/lib/audio-format.ts	-

8.3 Signierung

Signierte URLs werden clientseitig in src/lib/media.ts erzeugt und zwischengespeichert (inkl. Negativ-Cache für fehlende Objekte, 10 Minuten). Leserecht auf Objektebene entscheidet can_read_media().

8.4 Typische Fehler

Symptom	Ursache (typisch)	Prüfen
Bild lädt nicht, 400/404 auf der signierten URL	Objekt fehlt oder Variante nie erzeugt	Pfad in der Tabelle vs. <code>storage.objects</code> ; Backstop <code>ensureVariantsForPath</code>
Bild dauerhaft leer trotz vorhandenem Objekt	Leserecht (<code>can_read_media</code>)	Storage-Policies, Besitzpfad <code><userId>/...</code>
Video-Upload scheitert nach dem Hochladen	Containerprüfung fehlgeschlagen (Dauer/Größe/MIME)	Serverlogs von <code>registerVideoUpload</code> ; Original wurde bewusst gelöscht
Uploads generell nicht möglich	Session fehlt (401) oder Bucket-Policy	Anmeldung, Storage-Policies
Sehr langsame Bildauslieferung	Storage/CDN	Abschnitt 9; historische Referenz: Einzelausreißer bis 10,5 s im Lasttest

8.5 Zusammenhang Storage ↔ Datenbank-Timeout

Storage-Leserechte werden über SECURITY-DEFINER-Funktionen (`can_read_media()`) und damit über die Datenbank entschieden. Ist die Datenbank überlastet oder antwortet langsam, äußert sich das als Medienfehler oder Timeout beim Signieren – obwohl der Speicher selbst gesund ist. Umgekehrt erzeugt der Variantenausfall (`ensureVariantsForPath`) zusätzliche Server- und Datenbanklast. Bei gleichzeitigen Medien- und Timeout-Fehlern deshalb immer zuerst die Datenbank bewerten.

9. PERFORMANCE-NOTFALL

9.1 Entscheidungsbaum

Frage	Ja → Bereich
Sind die HTML-Antworten schnell, nur die Oberfläche träge/hängt?	A) Frontend (Rendering, lange Aufgaben, Bundlegröße)
Ist die erste HTML-Antwort langsam oder 5xx, auch bei /?	B) SSR / Worker (<code>src/server.ts</code> , <code>src/start.ts</code>)
Sind Server-Funktionen langsam, direkte Datenzugriffe schnell?	C) Server-Funktionen
Sind die REST-Antworten der Cloud langsam über alle Tabellen?	D) Data API
Sind einzelne Abfragen langsam, andere schnell?	E) Datenbank (Index, Query)
Steigt die Verbindungszahl an, Fehler „too many connections“?	F) Pooler / Verbindungen
Nur Medien betroffen, Rest normal?	G) Storage / CDN
Daten korrekt, aber nicht live; Presence tot?	H) Realtime

9.2 Messquellen im Projekt

- `/admin/health`: Fehler pro Stunde, p95-Antwortzeit je Bereich, offene Vorfälle, Selbsttest und Alarmtest.
- `src/lib/runtime-metrics.server.ts`: aggregierte Zähler (Anfragen, gleichzeitig laufende Anfragen, Fehler, Dauer, Event-Loop-Verzögerung), abrufbar über `/api/public/cache-metrics` (Token-geschützt).
- `ops_events` mit `area=performance` und den Latenzbudgets aus `src/lib/ops-monitor.shared.ts`.
- Backend: Slow-Query-Ansicht, Verbindungszahlen.
- `src/lib/http-cache.server.ts`: öffentliche, nicht personalisierte Seiten werden aus dem Cache beantwortet – ein plötzlicher Einbruch kann auch ein Cache-Miss-Muster sein.

9.3 Historische Referenz (KEIN aktueller Fehler)

Lasttest vom 14.09.2026, **rein lesend**, gegen Production (`docs/LASTTEST_1000VU_2026-09-14.md`): 606.325 Anfragen in 22,5 Minuten, davon 15 Minuten bei 1.000 gleichzeitigen Nutzern; 0 Serverfehler, 1 Timeout, 0 Verbindungsabbrüche; bei 1.000 VU p50 52 ms / p95 98 ms / p99 223 ms; maximal 38 Datenbankverbindungen; 0 Sperren, 0 Deadlocks; Datenbestand vor und nach dem Test identisch. Diese Werte dienen als Vergleichsmaßstab, nicht als Zustandsaussage für heute.

10. LOG-ANALYSE

10.1 Welche Logs existieren und wo

Quelle	Ort	Inhalt
Ereignisse/Vorfälle	<code>/admin/health</code> (Tabellen <code>ops_events</code> , <code>ops_incidents</code>)	Bereich, Schweregrad, Umgebung, Fingerprint, Notiz

Quelle	Ort	Inhalt
Serverlogs	Lovable-Projekt → Serverlogs (Preview und Published getrennt)	<code>console.error</code> aus Server-Funktionen, SSR, Middleware
Fehler-Middleware	<code>src/start.ts</code> ([<code>errorMiddleware</code>])	unbehandelte Serverfehler; veraltete Client-Bundles nur als info
Stack-Rettung	<code>src/lib/error-capture.ts</code>	Stacktraces, die die Serverschicht sonst verschluckt
Datenbank/Auth	Lovable Cloud Backend-Logs	Auth-Ereignisse, Postgres-Fehler, langsame Abfragen
Zahlungen	Stripe-Dashboard	Ereignisse, Webhook-Zustellversuche
Nutzermeldungen	<code>/admin/feedback</code> , <code>/admin/reports</code>	Fehlerberichte, Meldungen
Registrierung	<code>registration_events</code> , <code>registration_health_checks</code> , <code>/admin/registration-check</code>	Abbruchursachen ohne personenbezogene Daten
Sicherheit	<code>account_security_events</code> , <code>admin_audit_log</code>	Kontosicherheit, Adminaktionen
Browser	Konsole der Vorschau/Live-Seite	[<code>social</code>] ..., [<code>market</code>] ..., Hydration-Warnungen

10.2 Besonders relevante Meldungen

Meldung	Bedeutung	Erster Schritt
permission denied for table X	fast immer fehlender GRANT, nicht fehlende Policy	GRANTs der Tabelle prüfen
permission denied for function X	EXECUTE fehlt oder Aufruf läuft mit falscher Rolle (z. B. als anon)	prüfen, ob der Serverpfad Service-Role nutzt
DatabaseTimeout / Statement-Timeout	Überlast oder fehlender Index	Slow Queries, Verbindungszahl
5xx auf allen Routen	SSR/Worker	Buildlog, [<code>errorMiddleware</code>], letzte Veröffentlichung
401	erwarteter Zugriffsschutz oder fehlendes Bearer-Token	wenn alle geschützten Funktionen 401 liefern: <code>src/start.ts</code> prüfen
409 mit <code>stale_client_bundle</code>	alter Tab ruft eine Funktions-ID aus einem früheren Build; Client lädt sich neu	kein Serverausfall
RLS-Denials (Massen-401 unangemeldet)	Zugriffsschutz greift wie vorgesehen	kein Fehler
Storage-Fehler (400/404 auf signierten URLs)	Objekt fehlt oder Leserecht	Abschnitt 8
Realtime-Fehler / Kanal schließt	Websocket/Realtime	Abschnitt 6.6
[<code>unenv</code>] ... not implemented / <code>__dirname</code> is not defined	Node-fremdes Paket in der Worker-Laufzeit	Abhängigkeit ersetzen

Keine Secrets, Tokens oder Nachrichteninhalte in Vorfallsnotizen aufnehmen – Nutzer nur über technische Kennungen benennen, und nur wenn nötig.

11. SECURITY-NOTFALL

Allgemeiner Ablauf:

Verdacht → nichts löschen → eingrenzen → betroffene Systeme benennen
→ Zugänge/Secrets absichern → Schwachstelle beheben → Auswirkung prüfen
→ dokumentieren

Lage	Vorgehen
Möglicher Secret-Leak	Betroffene Variablennamen bestimmen (z. B. Cron-Tokens, VAPID, <code>LOVABLE_API_KEY</code> , Stripe-Schlüssel/Webhook-Secret, <code>MASTER_ADMIN_PASSWORD</code>). Reihenfolge: neues Secret in der Plattform-Secret-Verwaltung setzen → veröffentlichen → externen Aufrufer (Cron/Webhook) umstellen → altes Secret entfernen. Niemals Werte in Dokumente, Tickets oder Logs schreiben.
Kompromittiertes Konto	<code>account_security_events</code> und Auth-Logs prüfen; Sitzungen beenden bzw. Passwort-Reset anstoßen; Adminrollen in <code>user_roles</code> gegenprüfen; Aktionen in <code>admin_audit_log</code> nachvollziehen. Sperren (<code>user_bans</code>) bleibt eine manuelle Entscheidung.
Ungewöhnliche DB-Zugriffe	Zeitfenster und Tabellen bestimmen; <code>ops_events</code> mit <code>area=security/auth</code> ; Serverlogs; prüfen, ob ein Serverpfad fälschlich als anon läuft.

Lage	Vorgehen
RLS-Fehler / unberechtigter Zugriff	Betroffene Tabelle und Policy benennen; Reproduktion lesend ; Behebung ausschließlich als neue Migration (GRANT/Policy), danach <code>bunx vitest run tests/rls-policy-contract.test.ts</code> und Sicherheits-Scan. Policies nicht aufweichen, um einen Fehler zu „umgehen“.
Verdächtige Transaktionen	<code>market_transaction_events</code> (append-only) und die Abfragen aus 7.3 lesen; keine Datensätze ändern oder löschen; Ergebnis dokumentieren.

Löschläufe (retention-run) bei laufendem Sicherheitsvorfall **nicht** auslösen – Spuren müssen erhalten bleiben. Datenschutzvorfälle zusätzlich nach `docs/RUNBOOK_INCIDENT.md` §7 behandeln; die rechtliche Bewertung ist ausdrücklich nicht Teil dieses Handbuchs.

12. DEPLOYMENT / ROLLBACK

12.1 Tatsächlicher Weg

1. Änderung in der Vorschau prüfen (Vorschau = Staging, **gleiche** Datenbank).
2. Freigabe-Gate: `bun run verify (scripts/verify.sh) → tsc --noEmit, eslint ., vitest run`; Datenbanktests nur mit gesetztem `PGHOST`, E2E nur bei laufendem Dev-Server.
3. Build: `vite build` (Vite 8, Nitro, Cloudflare-Worker; Server-Entry `src/server.ts` über `vite.config.ts`).
4. Veröffentlichen über Lovable Publish. Production serviert den veröffentlichten Stand, Vorschau den letzten Build.
5. Rauchtest nach Abschnitt 3, Schritt 9.

`.github/workflows/ci.yml` läuft **nur** manuell (`workflow_dispatch`) und ist laut eigenem Kommentar nicht Teil des Betriebs – GitHub dient als Versionssicherung. Verbindlich ist das lokale Gate.

12.2 Migrationen

Drizzle-Kit gegen `drizzle/migrations/*.sql`, Verbindung über `LOVABLE_DB_MIGRATION_URL`. Journal und Snapshots unter `drizzle/migrations/meta`. Neue Migration statt Änderung einer bestehenden.

12.3 Rollback-Möglichkeiten

Gegenstand	Vorhandenes Verfahren
Code	Git-Historie; zusätzlich datierte Sicherungen unter <code>.lovable/backup/<datum>-<zweck>/</code> und <code>backups/<datum>-<zweck>/</code> , Release-Schnappschuss unter <code>release/production-video-business-v1-2026-09-01-rebased/</code> . Einzelne Dateien zurückspielen, nie einen Ordner blind über <code>src/</code> kopieren.
Veröffentlichter Stand	Erneutes Veröffentlichen eines geprüften Codestands. Ein „Deployment zurückrollen“-Verfahren ist im Repository nicht beschrieben → NICHT DOKUMENTIERT / EXTERN ZU PRÜFEN (Plattformfunktion).
Datenbankschema	Kein automatisierter Rollback. Vorhanden ist eine manuelle Baseline-Sicherung: <code>migration/production/2026-09-07-hardening-01-05/rollback-production-db-baseline.sql</code> – ausdrücklich „NICHT AUSGEFÜHRT“, liegt bewusst außerhalb der Migrationsordner. Für andere Migrationsnummern existiert keine solche Datei → NICHT DOKUMENTIERT / EXTERN ZU PRÜFEN .
Daten	Plattform-Backups von Lovable Cloud. Kein eigenes Backup-Skript, keine Dumps im Repository. Wiederherstellungsprobe: <code>scripts/restore-test.sh</code> . Zeitpunkt und Umfang einer Rückspielung → NICHT DOKUMENTIERT / EXTERN ZU PRÜFEN .
Storage-Objekte	Kein Wiederherstellungsverfahren im Repository → NICHT DOKUMENTIERT / EXTERN ZU PRÜFEN .

12.4 Typische Deployment-Fehlerquellen

- Route oder Komponente importiert eine `*.server.ts` → Build bricht ab.
- Geschützte Server-Funktion im Loader einer öffentlichen Route → `Unauthorized` beim Prerender.
- Laufzeitcode auf Modulebene in einer `*.functions.ts` → `ReferenceError` erst zur Laufzeit.
- Node-only-Paket in der Worker-Laufzeit.
- Neue Tabelle ohne GRANT → „permission denied“ trotz Policy.
- Neue Produktionsdomain nicht in `PRODUCTION_HOSTS` → Sandbox-Zahlungsmodus.

13. DATENSICHERHEIT DER MASSNAHMEN

Maßnahme	Bewertung
Lesende Abfragen (SELECT), Log- und Ereignisansichten	● sicher
/admin/health öffnen, Selbsttest, Alarmtest	● sicher
Rauchtest über die Oberfläche	● sicher
Funktion gezielt abschalten (/admin/ads, /admin/livetest)	● sicher
Vorfall in ops_incidents.note dokumentieren, Status setzen	● sicher
Erneutes Veröffentlichen eines geprüften Codestands	● nur nach Prüfung (Gate bun run verify bestanden)
Abschluss einer Market-Transaktion über den regulären Serverpfad wiederholen	● nur nach Prüfung (Abfragen aus 7.3 zuerst)
Secret-Rotation	● nur nach Prüfung; Reihenfolge neu setzen → veröffentlichen → externe Aufrufer umstellen → alt entfernen
Neue Migration mit GRANT/Policy zur Behebung eines Rechtefehlers	● nur nach Prüfung; danach RLS-Vertragstests + Sicherheits-Scan
Einzelne Datei aus einer Sicherung zurückspielen	● nur nach Prüfung (Diff lesen)
Sitzungen beenden / Passwort-Reset anstoßen	● nur nach Prüfung
Direkte UPDATES auf Produktionsdaten (z. B. market_items.status, messages)	● nicht durchführen
DELETE in Produktionstabellen	● nicht durchführen
DROP / TRUNCATE / Schema-Änderungen von Hand	● nicht durchführen
RLS deaktivieren oder Policies aufweichen	● nicht durchführen
Rechte an anon/authenticated/public pauschal vergeben	● nicht durchführen
Migration-Rollback (Baseline-SQL) einspielen	● nicht ohne externe Prüfung und ausdrückliche Freigabe
Bestehende Migrationen bearbeiten oder löschen	● nicht durchführen
retention-run während eines Sicherheitsvorfalls auslösen	● nicht durchführen
Sicherheitsbefunde ohne Prüfung als „behoben“/„ignoriert“ schließen	● nicht durchführen
Lasttest mit Schreiblast gegen Production	● nicht durchführen (eine geteilte Datenbank)

14. RECOVERY-CHECKLISTEN

Jede Liste: SYMPTOM → PRÜFUNG → URSACHE EINGRENZEN → MASSNAHME → VERIFIKATION → DOKUMENTATION.

☐ **Komplette Nichterreichbarkeit (P0)** SYMPTOM: / liefert 5xx oder nichts, alle Routen betroffen. PRÜFUNG: / und /auth direkt aufrufen; Serverlogs des veröffentlichten Deployments; Buildlog der letzten Veröffentlichung; /admin/health (falls erreichbar). URSACHE EINGRENZEN: letzte Veröffentlichung? Fehler in src/server.ts/ src/start.ts? Cloud oder Auslieferung gestört? MASSNAHME: keine weitere Veröffentlichung auf Verdacht; Ursache beheben oder geprüften Codestand erneut veröffentlichen. VERIFIKATION: Rauchtest komplett. DOKUMENTATION: ops_incidents + Vorfallsdatei.

☐ **Login-Ausfall (P1-A)** SYMPTOM: Anmeldung schlägt für alle fehl. PRÜFUNG: Serverlogs [auth]; Auth-Logs im Backend; /admin/registration-check; registration_events. URSACHE EINGRENZEN: Auth-Dienst, E-Mail-Bestätigung, Turnstile-Schalter, fehlendes Bearer-Token (src/start.ts). MASSNAHME: kleinste Ursache beheben; keine Auth-Einstellungen blind ändern. VERIFIKATION: Testanmeldung, Session gesetzt, geschützte Funktion antwortet. DOKUMENTATION: ops_events (area=auth) + Notiz.

☐ **DB/API-Ausfall (P1-B)** SYMPTOM: leere Daten oder „permission denied“/Timeouts über viele Bereiche. PRÜFUNG: Meldung genau lesen (Tabelle vs. Funktion); Backend-Logs; Verbindungszahl; Slow Queries. URSACHE EINGRENZEN: fehlender GRANT, Policy, Überlast, falsche Rolle im Serverpfad. MASSNAHME: bei Rechtefehler neue Migration mit GRANT; bei Überlast Abschnitt 9. VERIFIKATION: betroffene Ansicht lädt; RLS-Vertragstests grün. DOKUMENTATION: ops_events (area=database) + Migrationsverweis.

☐ **Messenger-Ausfall (P1-C)** SYMPTOM: Nachrichten nicht ladbar oder nicht sendbar. PRÜFUNG: Konsole [social] ...; Rechte auf messages/conversation_members; RPC mark_conversation_read; Realtime-Status. URSACHE EINGRENZEN: Tabelle 6.6. MASSNAHME: gemäß Bereich; keine Handänderungen an messages. VERIFIKATION: Nachricht senden, Empfang beim Gegenüber, Lesestand wird gesetzt. DOKUMENTATION: ops_events (area=rpc/database).

☐ **Market-Ausfall (P1-D)** SYMPTOM: Artikel/Transaktionen fehlerhaft, Status inkonsistent. PRÜFUNG: Abfragen A/B/C aus 7.3; Serverlogs von market-tx.server.ts; market_transaction_events. URSACHE EINGRENZEN: Rechte/Service-Rolle, RPC-Fehler, Client-Cache. MASSNAHME: Abschluss über den regulären Serverpfad

wiederholen; **kein** manuelles Status-UPDATE. VERIFIKATION: Abfragen A und C liefern keine Zeilen; Artikel im UI korrekt. DOKUMENTATION: Vorfallsnotiz mit Vorgangs-IDs (keine personenbezogenen Daten).

□ **Storage-Ausfall (P1-E)** SYMPTOM: Bilder/Audio fehlen, Uploads scheitern. PRÜFUNG: signierte URL im Browser; Objekt vorhanden? Serverlogs `registerVideoUpload/ensureVariantsForPath`. URSACHE EINGRENZEN: Objekt fehlt, Leserecht (`can_read_media`), Session, Datenbanküberlast (8.5). MASSNAHME: Varianten-Backstop laufen lassen; Rechte nur per Migration ändern. VERIFIKATION: Medien in Feed, Profil, Chat und Market laden; Testupload gelingt. DOKUMENTATION: `ops_events` + Notiz.

□ **Realtime-Ausfall (P2-C)** SYMPTOM: Nachrichten erst nach Neuladen, Presence/Tippen tot. PRÜFUNG: Websocket in den Netzwerkwerkzeugen; Kanalgrenzen (60 Chats, 80 Peers); Realtime-Status im Backend. URSACHE EINGRENZEN: Verbindung, Kanalgrenze, RLS auf `messages`. MASSNAHME: keine Codeänderung im Notfall nötig, wenn Lesepfad funktioniert – Nutzer können weiterarbeiten; Ursache nachgehen. VERIFIKATION: Nachricht erscheint beim Gegenüber ohne Neuladen. DOKUMENTATION: Notiz mit Beobachtungszeitraum.

□ **Performance-Einbruch (P2-B)** SYMPTOM: dauerhaft hohe Antwortzeiten, Timeouts. PRÜFUNG: `/admin/health` (p95 je Bereich), `runtimeMetrics`, `Slow Queries`, Verbindungszahl. URSACHE EINGRENZEN: Entscheidungsbaum 9.1. MASSNAHME: nur gezielt (Funktion abschalten, Index als neue Migration nach Analyse). Keine spontanen Optimierungen in Production. VERIFIKATION: Kennzahlen wieder im Rahmen; Vergleich mit 9.3. DOKUMENTATION: Messwerte in der Vorfallsnotiz.

□ **Security-Vorfall (P1-S)** SYMPTOM: Secret-Leak, Fremdzugriff, RLS-Verdacht. PRÜFUNG: `account_security_events`, `admin_audit_log`, `ops_events` (area=security/auth), Auth-Logs. URSACHE EINGRENZEN: betroffene Tabellen, Objekte, Secrets, Endpunkte benennen. MASSNAHME: nichts löschen; Secrets rotieren; Sitzungen beenden; Schwachstelle per neuer Migration beheben. VERIFIKATION: RLS-Vertragstests + Sicherheits-Scan; Zugriff nicht mehr möglich. DOKUMENTATION: Vorlage `docs/RUNBOOK_INCIDENT.md` §4, Kennzeichnung „Security“.

□ **Fehlerhaftes Deployment** SYMPTOM: Fehler unmittelbar nach einer Veröffentlichung. PRÜFUNG: Buildlog, Gate-Ergebnis, `[errorMiddleware]`, Abschnitt 12.4. URSACHE EINGRENZEN: letzte Änderung isolieren. MASSNAHME: keine weitere Veröffentlichung; Ursache beheben, `bun run verify`, erneut veröffentlichen; alternativ geprüften Stand aus Git/Sicherung. VERIFIKATION: Rauchtest. DOKUMENTATION: Vorfallsnotiz mit Commit/Stand.

15. KONTAKT- UND VERANTWORTUNGSBEREICHE

Nur Platzhalter – vor der ersten Nutzung ausfüllen. Keine erfundenen Angaben.

Bereich	Verantwortlich	Erreichbarkeit	Bekannt aus dem Projekt
Betreiber	_(ausfüllen)_	_(ausfüllen)_	Angaben im Impressum: <code>src/lib/legal/company.ts</code>
Technischer Verantwortlicher	_(ausfüllen)_	_(ausfüllen)_	-
Hosting / Auslieferung	_(ausfüllen)_	_(ausfüllen)_	Lovable (Publish) + Cloudflare-Auslieferung
Datenbank / Backend	_(ausfüllen)_	_(ausfüllen)_	Lovable Cloud (Postgres, Auth, Storage)
Domain / DNS	_(ausfüllen)_	_(ausfüllen)_	<code>y-dude.com</code> , <code>www.y-dude.com</code> , <code>y-dude.lovable.app</code>
Zahlungsanbieter	_(ausfüllen)_	_(ausfüllen)_	Stripe (Promotions, Abos)
Security	_(ausfüllen)_	_(ausfüllen)_	Adminrolle in <code>user_roles</code> , <code>admin_owners</code>
Backup / Recovery	_(ausfüllen)_	_(ausfüllen)_	Plattform-Backups Lovable Cloud
Alarmkanal / Heartbeat	_(ausfüllen)_	_(ausfüllen)_	<code>OPS_ALERT_WEBHOOK_URL</code> , <code>OPS_HEARTBEAT_URL</code> – laut <code>docs/PHASE6_...</code> noch nicht eingerichtet
Nutzersupport	_(ausfüllen)_	Impressumsadresse	<code>/admin/feedback</code> , <code>/admin/reports</code>

Mindestens zwei Personen sollten die Adminrolle in `user_roles` besitzen.

16. BEKANNTE BESONDERE Y-DUDE-BEFUNDE (bekannt / getestet)

Historische, belegte Erkenntnisse. **Keiner dieser Punkte ist ein aktuell offener Fehler**, soweit nicht ausdrücklich anders vermerkt.

Befund	Stand	Aussage
Production-Lasttest, rein lesend	14.09.2026, docs/LASTTEST_1000VU_2026-09-14.md	606.325 Anfragen, 0 Serverfehler, 1 Timeout; bei 1.000 VU p50 52 ms / p95 98 ms / p99 223 ms; 38 DB-Verbindungen; 0 Locks/Deadlocks; Datenbestand unverändert. Schreiblast war nicht Teil des Tests.
Read/Write-Lasttest	14.09.2026, docs/LASTTEST_1000VU_READ_WRITE_2026-09-14.md	NICHT DURCHGEFÜHRT (BLOCKED) – keine beschreibbare Testumgebung; Vorschau und Production teilen eine Datenbank. Alle Schreibpfade unter Last bleiben ungemessen.
Latenzverhalten unter Last	14.09.2026	Antwortzeiten sanken mit steigender Last (Cache-Aufwärmung); kein Kipppunkt bis 1.000 VU. Einzelausreißer bis 10,5 s bei der Bildauslieferung – offenes Optimierungspotenzial.
Abgewiesene unangemeldete Abfragen	14.09.2026	148.804 erwartete 401 (RLS greift); 147.925 zusätzliche DB-Rollbacks. Ein vorgeschalteter Kurzschluss für offensichtlich unangemeldete Abfragen wäre Optimierungspotenzial.
last_read_at / Messenger-Lesestand	14.09.2026	Aktualisierung des Lesestands (conversation_members.last_read_at) war die langsamste Abfrage im Normalbetrieb (Mittel 11,5 ms, Max 690 ms). Kein Fehler, aber der erste Kandidat für Optimierung.
Market-Kaufabschluss atomar	Migration 0037, verifiziert 09/2026	market_complete_transaction setzt completed und sold in einer Transaktion mit Zeilensperre; Artikel-ID stammt aus dem Vorgang, nie vom Client; wiederholter Aufruf idempotent (already_sold). 13/13 Unit-Tests.
Duplicate-Schutz Market	20260824174558, 0037, Webhook	partieller UNIQUE-Index für offene Vorgänge, already_sold-Guard, UNIQUE(provider,event_id) im Webhook, append-only Ereignisprotokoll.
Abholcode entfernt	09/2026	Der Market-Flow ist vereinfacht (Abholung, keine In-App-Zahlung für Artikelkäufe); ein Abholcode wird nicht mehr an den Client ausgegeben.
Service-Role-Bindung	09/2026	Rechtefehler bei Market/Chat waren auf einen veralteten Serverschlüssel zurückzuführen (Serveranfragen kamen als anon an), nicht auf fehlende Migrationen. Lehre: bei „permission denied“ im Serverpfad zuerst die Schlüsselbindung prüfen.
Market-Feed Empty-State	09/2026	Artikel von gefolgten Verkäufern erscheinen auch ohne gespeicherte Suche; Regressionstest tests/market-feed-empty-state.test.ts.
RLS-Ergebnisse post_originals	14.09.2026	RLS aktiv, genau eine Leseregel (Eigentümer oder Admin), keine öffentliche Regel; ein älterer Platzhalterbefund („No finding — verified safe“) ist veraltet und gegenstandslos.
Offene Warnhinweise des Sicherheits-Scans	14.09.2026	Vier Hinweise der Stufe „Warnung“ (SECURITY-DEFINER-Funktion für angemeldete Nutzer ausführbar; market_transaction_secrets; Eigentümer-Kennungen im öffentlichen Kanalverzeichnis; Schreibrechte ohne Policy auf channel_categories) – bewusst akzeptiert, keine kritischen Befunde.
Abhängigkeits-Audit	09/2026	6 betroffene Pakete, alle transitiv/build- oder testbezogen, 0 echte Schwachstellen in ausgelieferten Artefakten; TanStack-Pakete selbst nicht verwundbar.

Befund	Stand	Aussage
Grants breiter als nötig	14.09.2026	Für einzelne Tabellen bestehen Schreibrechte für anon/authenticated ohne passende Policy – wirkungslos, aber Aufrümpotenzial.
Bekannte, veraltete Testerwartungen	09/2026	Zwei E2E-Fälle (Feed-Anker „Global“, Messenger-Overlay) und tests/alerting-delivery.test.ts schlagen aus Testgründen fehl – keine Produktregression.

17. CHANGE- UND INCIDENT-LOG

Bei jedem Vorfall eine Zeile ergänzen; ausführliche Beschreibung bei P0/P1 zusätzlich als Datei unter docs/incidents/JJJJ-MM-TT-kurzname.md. Keine personenbezogenen Daten, keine Secrets.

Datum	Uhrzeit (UTC)	Incident	Auswirkung	Ursache	Maßnahme	Recovery	Verantwortlicher	Follow-up

Änderungen an diesem Handbuch:

Datum	Version	Änderung	Verantwortlicher
14.09.2026	1.0	Erstfassung aus dem aktuellen Production-Codestand	_(ausfüllen)_

18. ABSCHLUSS

Dieses Handbuch beschreibt die real existierende Y-Dude-Production-Codebasis zum 14.09.2026. Es enthält keine hypothetischen Funktionen, keine Secrets und es wurden für seine Erstellung keine Änderungen an Production vorgenommen. Ergänzende Detailabläufe: docs/RUNBOOK_CRITICAL_OPS.md (Schritt für Schritt), docs/RUNBOOK_INCIDENT.md (Vorfall und Datenschutz), docs/ARCHITEKTUR.md (Architektur im Detail).

19. OFFENE DOKUMENTATIONSLÜCKEN

Aus dem Repository **nicht** eindeutig ermittelbar:

- Rollback eines veröffentlichten Stands** – kein Verfahren im Repository beschrieben (Plattformfunktion). → EXTERN ZU PRÜFEN.
- Datenbank-Wiederherstellung** – Backups liegen bei der Plattform; Umfang, Aufbewahrung, Wiederherstellungszeit und der genaue Ablauf sind nicht im Repository dokumentiert. → EXTERN ZU PRÜFEN.
- Storage-Wiederherstellung** – kein Verfahren für verlorene Objekte im Bucket media dokumentiert. → EXTERN ZU PRÜFEN.
- Schema-Rollback allgemein** – eine manuelle Baseline-Sicherung existiert nur für die Härtung vom 07.09.2026; für alle anderen Migrationsnummern gibt es kein Rückwärtsverfahren.
- Alarmkanal und Heartbeat** – OPS_ALERT_WEBHOOK_URL, OPS_ALERT_WEBHOOK_URL_2, OPS_HEARTBEAT_URL sind im Code vorgesehen; laut docs/PHASE6_BETRIEB_AUSFALLSICHERHEIT_2026-08-27.md noch nicht eingerichtet. Ob sie inzwischen gesetzt sind, ist aus dem Repository nicht feststellbar.
- Kontakt- und Verantwortungsdaten** (Abschnitt 15) – nicht im Repository vorhanden, bewusst als Platzhalter geführt.
- Stripe-Secret-Namen** – der Zahlungs-Webhook prüft die Signatur über src/lib/stripe.server.ts; die vollständige Liste der dort gelesenen Umgebungsvariablenamen wurde für dieses Handbuch nicht abschließend verifiziert.
- Divergenz bei Cron-Auth** – src/routes/api/public/moderation-run.ts nutzt eine eigene Vergleichsfunktion und nur MODERATION_CRON_TOKEN, während alle anderen Endpunkte den gemeinsamen Helfer src/lib/worker-auth.server.ts verwenden. Ob dies Absicht oder Altbestand ist, ist nicht dokumentiert.
- Externe Zeitplan-Konfiguration** – die Cron-Jobs stehen in der Datenbank; welche Adresse (Production oder Vorschau) sie aufrufen und wo diese Konfiguration verwaltet wird, ist im Repository nicht festgehalten.
- Schreiblastverhalten unter Last** – Messenger-Schreiblast, Market-Abschlüsse und Uploads sind unter Last nie gemessen worden (siehe Abschnitt 16, BLOCKED). Aussagen dazu wären unbelegt.
- Speicherbereiche über media hinaus** – in der Live-Instanz existiert genau ein Bucket; ob weitere Bereiche geplant oder extern verwaltet sind, ist nicht dokumentiert.
- Fehlerbudget / Zielwerte (SLO)** – Latenzbudgets stehen im Code (src/lib/ops-monitor.shared.ts); verbindliche Betriebsziele (Verfügbarkeit, Wiederherstellungszeit) sind nicht festgelegt.

